

Arboles AVL

Un árbol AVL es un tipo especial de árbol binario ideado por los matemáticos soviéticos Adelson-Velskii y Landis.

Los **árboles AVL** son árboles BB donde todo nodo cumple la propiedad de equilibrado AVL:

- La altura del subárbol izquierdo y del derecho no se diferencian en más de uno.

Se define **factor de equilibrio** de un nodo como:

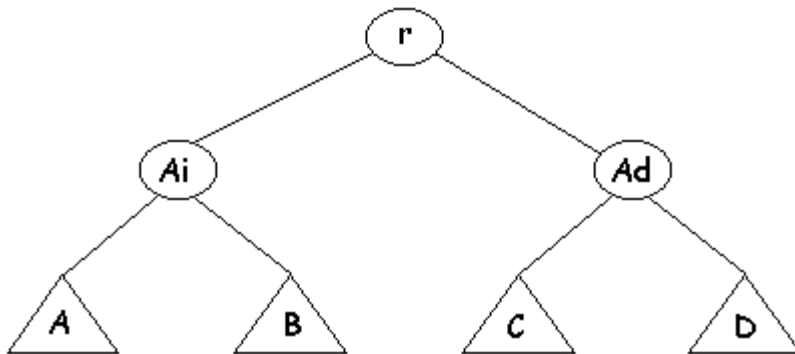
$$Fe(\text{nodo}) = \text{altura}(\text{derecho}) - \text{altura}(\text{izquierdo})$$

En un árbol AVL el factor de equilibrio de todo nodo es -1, 0 ó +1.

Tras la inserción o borrado de un elemento, sólo los ascendientes del nodo pueden sufrir un cambio en su factor de equilibrio, y en todo caso sólo en una unidad.

Se añade una etapa donde se recorren los **ascendientes**. Si alguno está desequilibrado (+2 o -2) se vuelve a equilibrar mediante operaciones denominadas **rotaciones**.

Estructura:



Esquema general de árbol AVL.

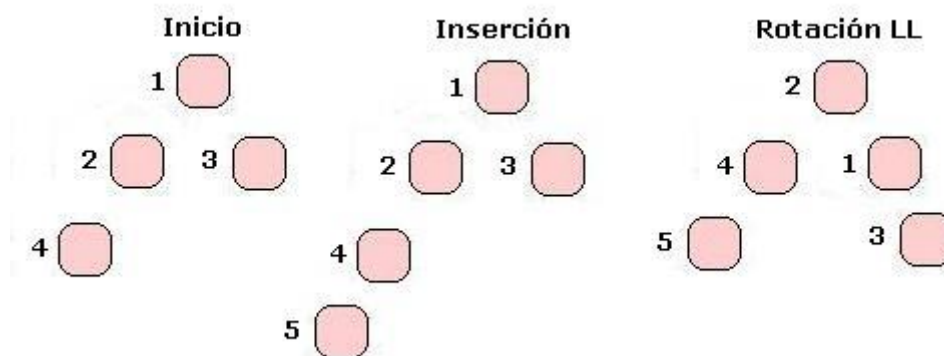
¿Por qué se dicen que están balanceados?

Su principal característica es la realización de reajustes o balanceos después de inserciones o eliminaciones de elementos. La propiedad de equilibrio que debe cumplir un árbol para ser AVL asegura que la profundidad del árbol sea $O(\log(n))$, por lo que las operaciones sobre estas estructuras no deberán recorrer mucho para hallar el elemento deseado. El tiempo de ejecución de las operaciones sobre estos árboles es, a lo sumo $O(\log(n))$ en el peor caso, donde n es la cantidad de elementos del árbol. El balance de un nodo en un árbol binario en general y de un árbol AVL en particular, se define como la altura de su subárbol izquierdo menos la altura de su subárbol derecho: $|\text{altura}(\text{arbolIzquierdo}) - \text{altura}(\text{arbolDerecho})| < 2$ También existe la correspondiente definición simétrica: la altura de su subárbol derecho menos la altura de su

subárbol izquierdo; pero en este blog se adoptará la primera. Por convención, la altura de un árbol AVL nulo o vacío se define como -1. Durante el proceso de inserción o eliminación puede ocurrir un desbalance, es decir, que el valor absoluto de la altura de alguno de los nodos del árbol AVL sea mayor que 1.

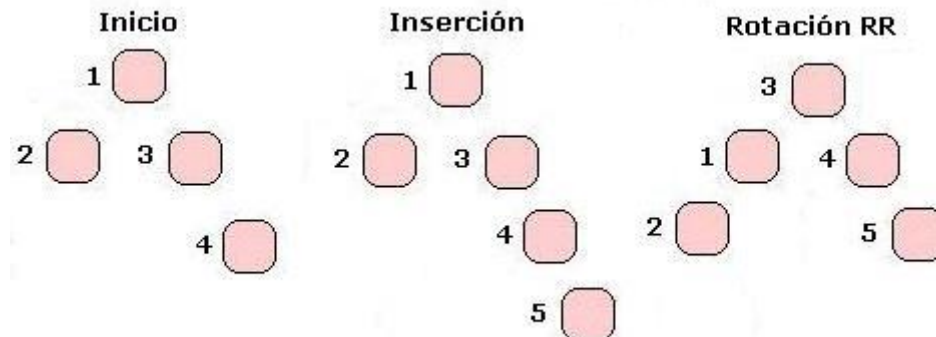
¿Cómo se balancea un Árbol AVL tras las operaciones de inserción?

Para balancear el árbol deberemos tomar el hijo izquierdo de 1 (llamémoslo 2), y ponerlo como padre de 1, el hijo derecho de 1 queda en su lugar, el hijo izquierdo de 2 queda en su lugar y el hijo de derecho de 2 debe ir como hijo izquierdo de 1, ya que en su lugar tendremos a 1. Veamos un dibujo:



Lo que hacemos es mover el subárbol que queda con un largo excesivo hacia arriba, sacando al padre para hacer lugar, teniendo cuidado eso si de mantener la relación de mayor menor, lo cual hacemos dejando los subárboles izquierdos de 2 y derecho de 1 en sus lugares (ya que son menores que todos los demás, y mayores respectivamente), y moviendo de lugar el subárbol derecho de 2 hacia el izquierdo de 1. Esto lo podemos hacer ya que este subárbol contiene elementos mayores que los de 2, pero menores que 1.

En el caso de la rotación simple RR sucede lo mismo que acabamos de ver, solo que en el caso de insertar a la derecha del hijo derecho:



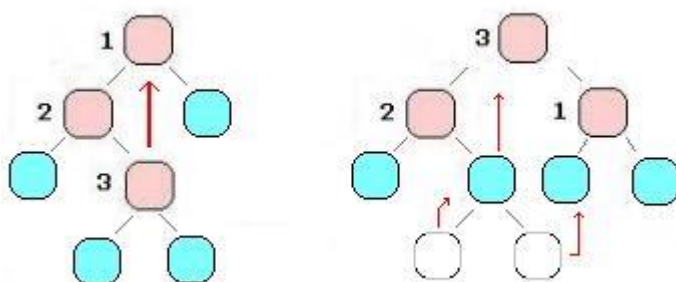
Hemos visto los dos casos simples, donde insertamos en la rama menor que todos o mayor que todos. Veremos ahora que sucede cuando insertamos en el hijo derecho del subárbol izquierdo, o en el izquierdo del derecho, es decir cruzado.



En este caso lo que haremos será aplicar dos rotaciones simples, con lo cual haremos que el nodo insertado «suba» acortando la altura del árbol. Noten que el nodo que tiene $FB = 2$ es el nodo 1, y no el nodo 2 que es donde insertamos (en su subárbol derecho). Por lo cual deberemos modificar la altura de sus hijos para reducir el FB a 1.

El proceso que seguiremos será el siguiente, tomamos el nodo en la rama donde se generó el desbalance, en este caso el 2, como la inserción es en el hijo derecho de 2, tomamos este nodo (el 3), y lo guardamos como la nueva raíz. Guardamos los dos hijos del 3 (ya que en realidad el árbol puede ser más amplio, y haber nodos hacia «abajo») y ponemos como su hijo izquierdo a 2, y su hijo derecho a 1. Así sabemos que el orden se mantiene, 3, estaba a la derecha de 2, por lo que es mayor, y a la izquierda de 1, por lo que es menor. Colocado 3 en el lugar de 1, tomamos los dos posibles hijos de 3 que guardamos y los colocamos respectivamente como hijo derecho de 2 al hijo izquierdo, y como hijo izquierdo de 1 al derecho de 3. Otra vez manteniendo el orden.

Veamos un ejemplo gráfico:



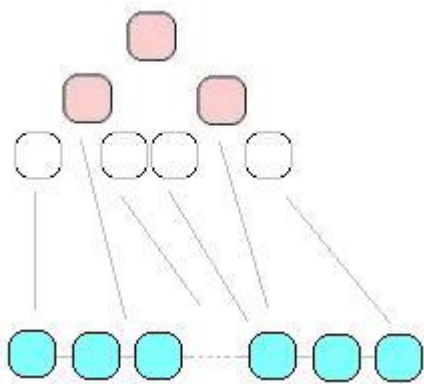
En resumen, la idea es, cuando detectamos que un nodo tiene $FB = 2$ (recuerden, en valor absoluto), entonces de acuerdo con sea positivo o negativo tomamos la rama izquierda o la derecha, subimos el nodo correspondiente y reasignamos con cuidado los hijos para no perder el orden. Las operaciones LR y RL, como las LL y RR son simétricas, por lo que el código será idéntico cambiando el hijo seleccionado. Como mencionábamos antes, las operaciones complejas se

pueden reducir a dos operaciones simples, por ejemplo, la LR podemos reducirla a realizar primero una RR en el hijo izquierdo y luego una LL en la raíz.

Al insertar buscamos la posición del elemento como lo hacemos normalmente en un árbol binario, buscando en derecho o izquierda de acuerdo con la comparación. Una vez que encontramos su posición creamos un nuevo nodo y lo insertamos, al regresar de la recursión debemos balancear los nodos ya que podríamos haber creado un desbalance, actualizando la altura según corresponda.

Con esto completamos nuestro TAD AVL, como ya dijimos tenemos la capacidad de ordenar el árbol y asegurarnos que el orden de búsqueda de un elemento no supere nunca $\log(n)$. Un buen ejercicio de multiestructuras es pensar en la combinación de un AVL con una lista, de forma que podamos buscar determinado valor dentro del AVL, y luego a partir de ese valor podamos recorrer una cantidad de valores ordenados sin tener que buscar cada uno.

Supongamos que tenemos tiempos ordenados en un AVL, y queremos obtener los 10 tiempos siguiente a partir de cierto momento. Entonces lo que podemos hacer es tener un AVL «espejado» en una lista, con lo cual podemos buscar el tiempo deseado en el AVL (o el más cercano a él), y luego de encontrar el mismo saltamos a la lista (mantendremos un puntero a la posición correspondiente en cada nodo del AVL) y recorreremos la lista obteniendo los 10 elementos. Así logramos un buen grado de eficiencia, teniendo únicamente que buscar un tiempo en el AVL con un orden $\log(n)$ y luego recorriendo la lista que ya sabemos esta ordenada ya que mantenemos el orden al hacer cada inserción.



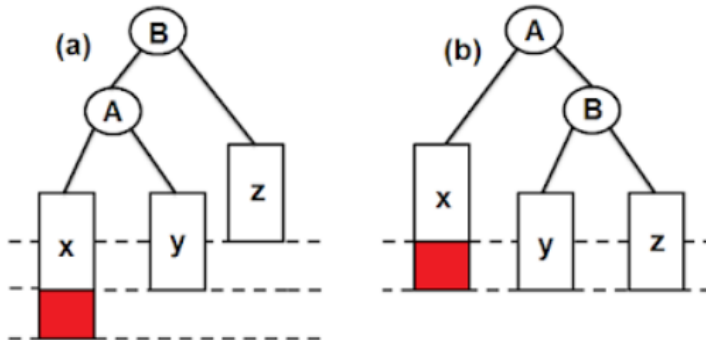
¿Cuáles son los casos de desbalanceo y cómo se solucionan?

En este sentido, existen cuatro casos que corrigen el balanceo de un árbol AVL:

1. **Caso 1:** rotación simple derecha.
2. **Caso 2:** rotación simple izquierda.
3. **Caso 3:** rotación doble izquierda derecha.
4. **Caso 4:** rotación doble derecha izquierda.

Rotación simple.

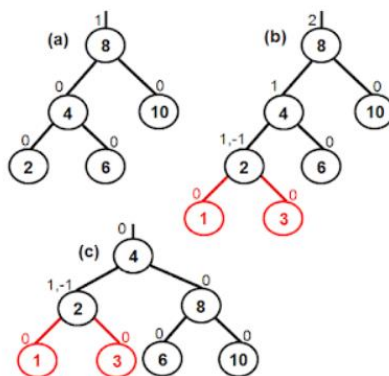
El primer caso de rebalanceo, la rotación derecha también conocida como rotación simple, se ilustra en la siguiente figura:



La figura (a) muestra hasta antes de la inserción del elemento representado por el cuadrado rojo (cuadrado más oscuro), un árbol AVL balanceado. El balance puede determinarse gráficamente por los niveles representados por líneas horizontales punteadas: el balance para los nodos A y B es 0 y 1 respectivamente. Al insertar el elemento representado por el cuadrado rojo (cuadrado más oscuro), se presenta un desbalance sobre el nodo B: ahora el balance para los nodos A y B es 1 y 2 respectivamente.

Por otro lado, la figura (b) muestra la solución al desbalanceo descrito en el párrafo anterior. Para visualizarlo mejor, imagine que en la figura (a), en el nodo representado por B existe una polea fija, de tal forma que al "jalar" z hacia abajo, el subárbol izquierdo de B representado por A sube, mientras que B baja convirtiéndose en el subárbol derecho de A. Note cómo y pasa de ser subárbol derecho de A, a ser subárbol izquierdo de B.

Para el caso de la rotación izquierda el proceso es análogo pero de manera simétrica, al de la rotación derecha descrita en esta sección.



Ejemplo de la aplicación de la rotación simple.

La rotación simple derecha se presenta al insertar los elementos 1 o 3, los cuales se resaltan y presentan en la figura (b).

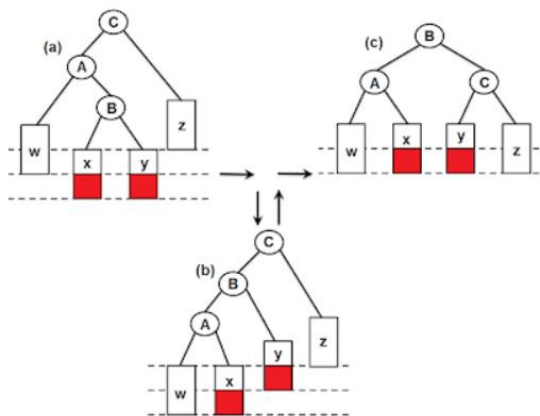
El nodo desbalanceado es el que contiene al elemento 8 (¿Por qué?). La aplicación del Caso 1 de rotación dará como resultado el árbol AVL que aparece en la figura (c) dependiendo del elemento (1 o 3) que se haya insertado, de tal forma que la rotación simple derecha se realiza sobre el nodo que contiene al elemento 8 de la figura (b).

Observe cómo la aplicación de la rotación simple corrige el balance general del árbol, de tal forma que el balance obtenido es casi perfecto.

Antes de continuar, asegúrese de comprender el proceso de rebalanceo aplicado a la figura anterior, así como de determinar por su propia cuenta, el balance de cada uno de los nodos para los tres incisos.

Rotación doble.

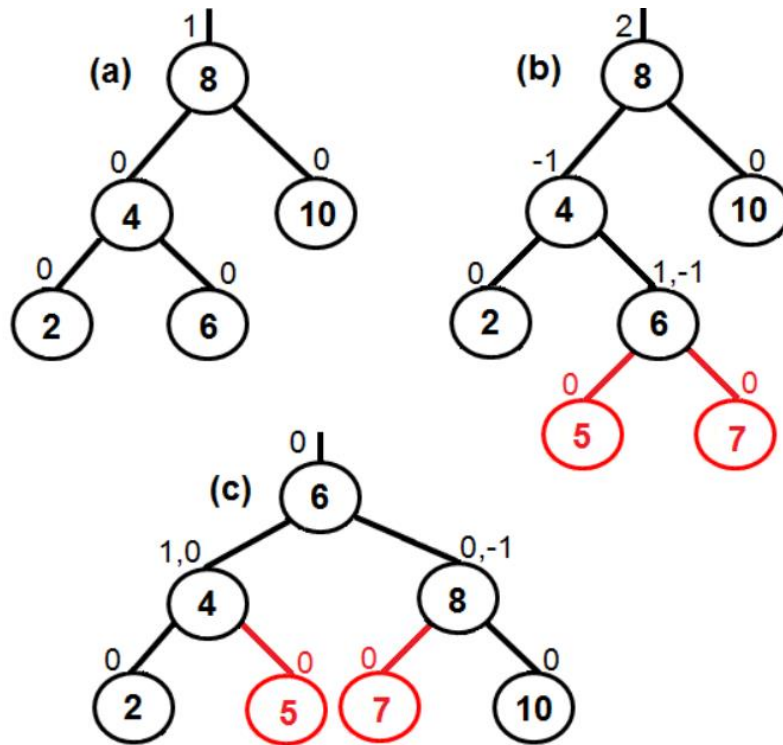
El caso de rebalanceo que implica una rotación doble es un proceso un poco más elaborado que el de la rotación simple, ya que como su nombre lo indica, implica dos rotaciones. La rotación doble izquierda derecha se muestran en la siguiente figura:



Caso3: rotación doble izquierda derecha (adaptada de Wirth).

- La figura (a) muestra, hasta antes de la inserción del elemento representado por el cuadrado rojo (cuadrado más oscuro), un árbol AVL balanceado. El balance puede determinarse gráficamente por los niveles representados por líneas horizontales punteadas: el balance para los nodos A, B y C es 0, 0 y 1 respectivamente. Al insertar cualquiera de los elementos representados por el cuadrado rojo (cuadrado más oscuro), se presenta un desbalance sobre el nodo C: ahora el balance para los nodos A, B y C es -1, (1 o -1 según sea el elemento insertado) y 2 respectivamente.
- Aunque C es el nodo desbalanceado, observe que el balance no se corrige si se realiza una rotación derecha sobre C (asegúrese de comprobarlo). Por otro lado, note los signos de los balances generados por la inserción del elemento que provoca el desbalance.
- En base a lo anterior, la figura (b) muestra una rotación izquierda sobre A aunque el nodo A no está desbalanceado. Note que el balance no se ha corregido todavía, ya que el balance de A es 0 o 1, el de B es 2 o 1, y el de C 2.

- Ahora bien, partiendo de la figura (b), una nueva rotación derecha sobre C generará el árbol de la figura (c) mejorando significativamente el balance general de la mayoría de los nodos.



La rotación doble se presenta al insertar cualquiera de los elementos 5 o 7 (figura (b)).

La aplicación de la rotación doble dará como resultado el árbol AVL que aparece en la figura (c) dependiendo del nodo que se haya insertado. Se deja como ejercicio para el lector generar el paso intermedio y comparar su resultado con el de la figura (c). El paso intermedio consiste en hacer una rotación izquierda sobre 4 en la figura (b), y posteriormente una rotación derecha sobre 8; estas dos rotaciones deberán generar la figura (c). Una vez más observe los signos de los balances en la rotación simple y en la doble; y asegúrese de comprender los procesos de balanceo aplicados en los ejemplos.

Bibliografías

<https://estructurasite.wordpress.com/arbol-avl/#:~:text=Un%20%C3%A1rbol%20AVL%20es%20un,una%20etapa%20posterior%20de%20reequilibrado.>

<https://www.wanderingbit.com/tag/balanceo-de-avl/#:~:text=Para%20balancear%20el%20%C3%A1rbol%20deberemos,su%20lugar%20tendremos%20a%201.>

<http://fundamentospoorrr.blogspot.com/2017/06/arboles-binarios-balanceados-avl.html>